Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

High-Performance Computing Lab for CSE **2026**

Student: Riccardo Polo Discussed with: Giovanni Guidarini

Solution for Project 6

Due date: **Friday** 22 May 2026, 18:00 (Europe/Zurich)

HPC Lab for CSE 2025 — Submission Instructions

Following these instructions is mandatory: Submissions that do not comply **will not be considered**.

- **Submission Platform**: Assignments must be submitted via SAM-UP.
- **Required Files**: Your submission must include:
  - **Report**: A PDF summarizing your results and observations, based on this LaTeX template.
    - File name: `project_number_lastname_firstname.pdf`
    - You are allowed to discuss all questions with anyone you like; however: (i) your submission must list anyone you discussed problems with and (ii) you must write up your submission independently.
  - **Source Code Archive**: A **compressed tar archive**, named `project_number_lastname_firstname.tar.gz`, containing:
    - All source code files.
    - Build files and scripts (e.g., `Makefile`, batch job scripts, plot scripts, etc). If you modified any provided files, ensure they still work as expected.
    - The full LaTeX source of your report including all needed files to build the report (figures, listings, etc).
  - **Size Limit**: The total submission (PDF + archive) must not exceed **25 MB**.
  - **File Naming Rules**: Use only `ASCII` characters (avoid spaces, special symbols, and non-English letters).
- **Grading Process**: The TAs will evaluate your project based on:
  - Your report write-up.
  - Your code implementation.
  - Benchmarking your code's performance.

1. Graph Partitioning with Julia: Load balancing for HPC [50 points]

1.1. Graph Partitioning Algorithms

1.1.1. Inertial Bisection algorithm

Listing 1 reports the implementation of the inertial bisection algorithm. This method uses the geometrical information associated with the graph vertices, namely their coordinates, to split the graph into two subdomains. The basic idea is to interpret the vertices as a set of points in the plane and to compute the direction along which the point cloud has the largest spatial spread. The graph is then partitioned by projecting the vertices onto this direction and separating them according to their position with respect to a cutting line.

Listing 1: Inertial Bisection algorithm

```
function inertial_part(A, coords)

    p = ones(Int, size(coords, 1))

    xbar = mean(coords[:, 1])
    ybar = mean(coords[:, 2])

    Sxx = sum((coords[:, 1] .- xbar) .^ 2)
    Syy = sum((coords[:, 2] .- ybar) .^ 2)
    Sxy = sum((coords[:, 1] .- xbar) .* (coords[:, 2] .- ybar))

    M = Symmetric([Syy -Sxy;
                   -Sxy Sxx])

    eig = eigen(M)
    eigv = eig.vectors[:, argmin(eig.values)]

    p1, p2 = partition(coords, eigv)

    p[p1] .= 1
    p[p2] .= 2

    return p
end
```

The function first initializes the partition vector `p`, assigning all vertices to the same subdomain. The number of entries of `p` is equal to the number of rows of `coords`, hence to the number of vertices of the graph.
The quantities `xbar` and `ybar` represent the coordinates of the center of mass of the vertices. The coordinates are then shifted with respect to this center of mass in order to compute the second-order moments `Sxx`, `Syy`, and `Sxy`. These quantities measure the spatial distribution of the vertices around the center of mass: `Sxx` and `Syy` quantify the spread along the coordinate directions, while `Sxy` measures the correlation between the two coordinates.

Using these moments, the matrix

$$M = \begin{pmatrix} S_{yy} & -S_{xy} \\ -S_{xy} & S_{xx} \end{pmatrix}$$

is assembled as a symmetric matrix. The eigenvector associated with the smallest eigenvalue of `M` minimizes the sum of squared distances of the vertices from the corresponding line. Equivalently, the resulting line follows the direction of largest spatial spread of the point cloud. Since `M` is only a 2×2 dense symmetric matrix, the standard dense routine `eigen` is sufficient and avoids unnecessary use of ARPACK on such a small problem. The selected eigenvector is stored in `eigv`.
The actual split is then performed by the auxiliary routine `partition(coords, eigv)`. This routine projects the vertex coordinates onto the inertial direction and separates the vertices into two sets, denoted by `p1` and `p2`. These two sets correspond to the two sides of the bisecting line. Finally, the partition vector is updated by assigning label `1` to all vertices in `p1` and label `2` to all vertices in `p2`. The returned vector `p` therefore contains the bisection of the graph induced by the inertial direction.

1.1.2. Spectral Bisection Algorithm

In Listing 2, we report the implementation of the spectral bisection algorithm. Unlike inertial bisection, this method does not use the geometrical coordinates of the vertices. Instead, it exploits the connectivity structure of the graph through the graph Laplacian matrix. The main idea is to compute an eigenvector of the Laplacian, known as the Fiedler vector, and use its components to divide the vertices into two groups.

Listing 2: Spectral Bisection algorithm

```
function spectral_part(A)

    n = size(A, 1)

    if n == 1
        return ones(Int, 1)
    elseif n == 2
        return [1, 2]
    end

    # 1. Construct the sparse graph Laplacian.
    d = vec(sum(A, dims=2))
    L = spdiagm(0 => d) - A

    # 2. Compute the Fiedler vector.
    if n <= 4
        eig = eigen(Symmetric(Matrix(L)))
        perm = sortperm(eig.values)
        fiedler_vec = eig.vectors[:, perm[2]]
    else
        eigvals, eigvecs = eigs(L; nev=2, which=:SM)
        perm = sortperm(real(eigvals))
        fiedler_vec = real.(eigvecs[:, perm[2]])
    end

    # 3. Balanced partition by sorting the Fiedler vector.
    order = sortperm(fiedler_vec)

    p = ones(Int, n)

    n1 = div(n, 2)
    p[order[1:n1]] .= 1
    p[order[n1+1:end]] .= 2

    return p

end
```

The function takes as input the adjacency matrix `A` of the graph and first extracts the number of vertices `n`. A special case is included for graphs with a single vertex: in this situation no meaningful bisection can be performed, and the function simply returns a partition vector containing one label.

The first step of the algorithm consists in constructing the graph Laplacian. The vector `d` contains the degrees of the vertices, computed by summing the rows of the adjacency matrix. The Laplacian is then assembled in sparse form as

$$L = D - A.$$

Keeping `L` sparse is important because the adjacency matrices of mesh graphs are sparse, and forming a dense matrix would destroy this structure unnecessarily.

The spectral information of the graph is then obtained from the eigenvectors of `L`. For the general case, the implementation uses `eigs` from ARPACK to compute only the two eigenpairs of smallest magnitude. This is sufficient because the first eigenvector of the Laplacian corresponds to the constant mode, while the second eigenvector is the Fiedler vector used for the bisection. A small dense fallback is used only for very

small subgraphs, where ARPACK is not appropriate.

In the theoretical version of spectral bisection, the split can be obtained by comparing the components of the Fiedler vector with a threshold, often chosen as zero or as the median value. In this implementation, a balanced partition is enforced explicitly. The entries of the Fiedler vector are sorted, and the first `div(n, 2)` vertices in this ordering are assigned to the first subdomain, while the remaining vertices are assigned to the second subdomain. This guarantees that the two parts have approximately the same number of vertices, which is desirable in recursive graph partitioning because it avoids strongly unbalanced subdomains.

Finally, the partition vector `p` is returned. Its entries contain the labels of the two subdomains: vertices assigned label `1` belong to the first part, while vertices assigned label `2` belong to the second part. In disconnected or nearly disconnected subgraphs, the Fiedler vector may not be unique; therefore, the exact shape of the spectral partition can be sensitive to eigenvalue degeneracies. This is one reason why the partitions are inspected together with their edge-cuts and part sizes.

1.1.3. Results

The script `main_bench.jl` was used to run the four bisection strategies on the toy meshes loaded by `bench_bisection.jl`. For each mesh and each partitioning strategy, we measured the bisection edge-cut, i.e. the number of graph edges whose endpoints belong to different parts. Since the partitions are constrained to be balanced, the edge-cut provides a simple measure of the quality of the interface between the two subdomains: lower values correspond to fewer inter-partition connections and therefore to less communication in a parallel graph-distributed computation.

Mesh	Coordinate	Inertial	Spectral	METIS
`3elt`	–	–	–	–
`airfoil1`	–	–	–	–
`barth4`	–	–	–	–
`crack`	–	–	–	–
`mesh1e1`	–	–	–	–
`mesh2e1`	–	–	–	–
`mesh3e1`	–	–	–	–
`netz4504_dual`	–	–	–	–
`stufe`	–	–	–	–
`ukerbe1`	–	–	–	–

Table 1: Bisection edge-cut obtained on the toy meshes loaded by `bench_bisection.jl`.

The corresponding partitions are shown in Figures 1–4 for the first four meshes, in Figures 5–9 for the additional structured meshes, and in Figure 10 for the `ukerbe1` mesh. The coordinate and inertial methods are purely geometrical: they only use the position of the vertices and therefore tend to produce almost straight cuts. This behaviour is clearly visible in the `3elt`, `barth4`, and `crack` examples. Such partitions are simple and often reasonable when the mesh is approximately uniform and the graph connectivity follows the geometry closely. However, they can become sub-optimal when the mesh has non-uniform density or when the geometry contains local features that should not be cut through.

The spectral and METIS partitions are instead based on the graph connectivity. As a consequence, they are generally more flexible: the interface is not restricted to be a straight geometrical line, but can follow weaker connections in the graph. This is particularly important for meshes such as `airfoil1`, where the local refinement around the airfoil and its wake makes a purely geometrical split less adapted to the graph structure. In this case, the coordinate and inertial cuts separate the mesh in a rather direct left-right fashion, while spectral bisection and METIS produce more irregular interfaces that better reflect the connectivity of the mesh.

Some partitions may appear visually surprising. In particular, for meshes such as `barth4`, `3elt`, and `ukerbe1`, the spectral and METIS partitions sometimes contain one part that occupies a relatively small physical region of the plotted domain. This should not immediately be interpreted as poor load balance. The balance constraint is imposed on the number of graph vertices, not on the physical area covered by each part. Therefore, in highly refined regions, a geometrically small subdomain may still contain a large number of vertices. For this reason, the visual inspection of the partitions should always be complemented by the edge-cut values and, when needed, by the number of vertices assigned to each part. This distinction

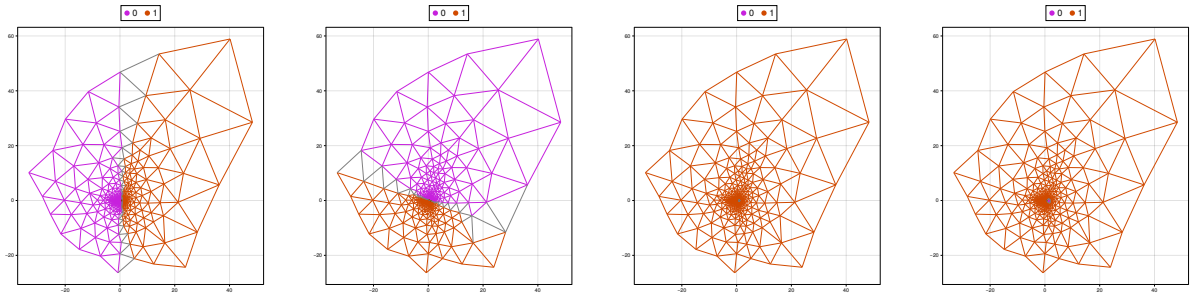
is particularly important for non-uniform meshes, where physical area and graph size are not proportional.

The additional structured meshes further illustrate the difference between geometrical and connectivity-based partitioning. For the regular meshes `mesh1e1`, `mesh2e1`, and `mesh3e1`, all methods produce relatively clean partitions, but the shape of the interface depends on the criterion used by the algorithm. Coordinate bisection typically produces a vertical or horizontal cut aligned with the coordinate axes, while inertial bisection may rotate the cut according to the dominant spatial direction of the mesh. Spectral bisection and METIS may instead introduce slightly irregular interfaces when this reduces the number of cut edges.

The `netz4504_dual` and `stufe` meshes are more interesting because their geometry is less uniform. In these cases, a purely geometrical cut does not necessarily correspond to the smallest communication interface. The graph-based methods are able to adapt the separator to the local connectivity of the mesh, especially around refined or stepped regions. This is the expected behaviour: the objective of graph partitioning is not to produce a visually straight cut, but to minimize the number of edges crossing between different parts while keeping the load balanced.

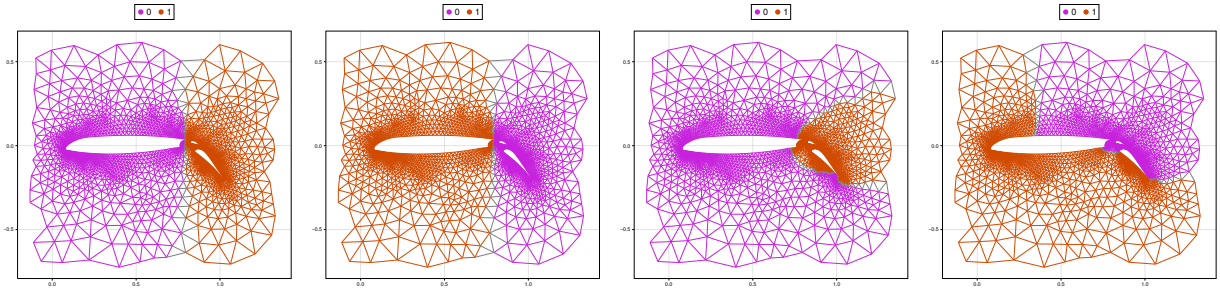
Finally, the `ukerbe1` mesh shows a geometry with a pronounced notch-like feature and several internal circular holes. In this case, the coordinate and inertial methods still produce mostly straight cuts, separating the mesh approximately into upper and lower parts. This is reasonable from the point of view of the global geometry, but it does not explicitly account for the connectivity around the notch and the locally refined region. The spectral and METIS partitions instead place the separator close to this narrow central region. This behaviour is consistent with the graph-based objective, since cutting through a geometrical bottleneck can reduce the number of edges crossing between the two parts. Although some of these graph-based partitions may occupy a small physical area, this is not necessarily a sign of poor balance, because the local mesh density near the notch is much higher than in the coarser parts of the domain.

Overall, the results confirm the expected trade-off between geometrical and graph-based partitioning. Coordinate and inertial bisection are cheap and produce simple cuts, but they may ignore important connectivity features. Spectral bisection and METIS can produce lower edge-cuts because they exploit the graph structure directly, but the resulting partitions may be less regular geometrically and, in some cases, should be inspected to ensure that the balance constraint has been respected.



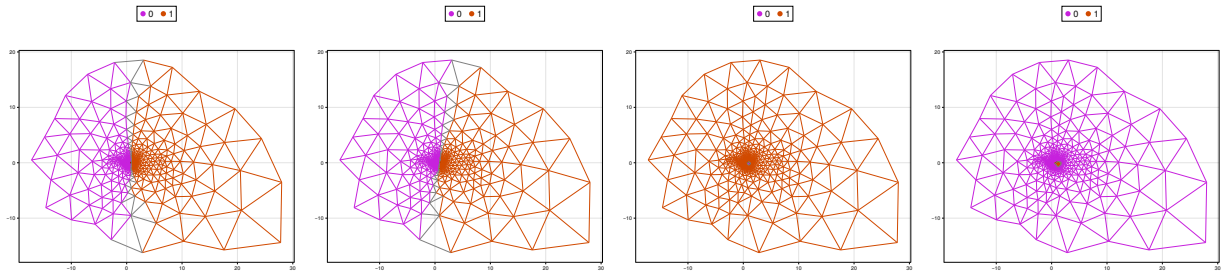
(a) Coordinate (b) Inertial (c) Spectral (d) METIS

Figure 1: Bisection partitions obtained for the `3elt` mesh.



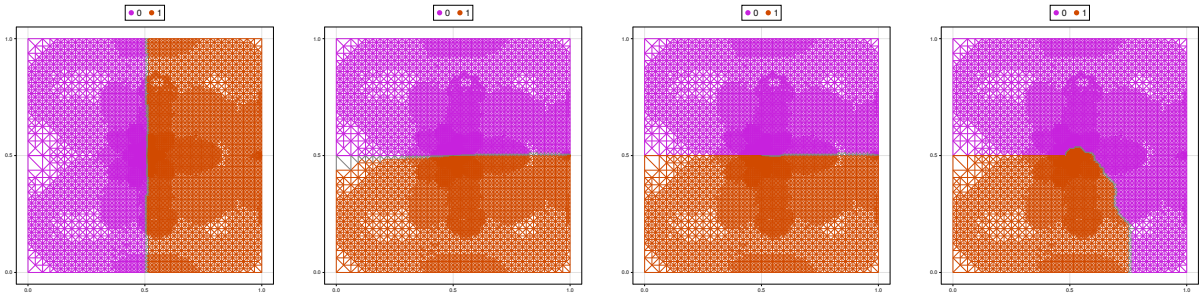
(a) Coordinate (b) Inertial (c) Spectral (d) METIS

Figure 2: Bisection partitions obtained for the `airfoil1` mesh.



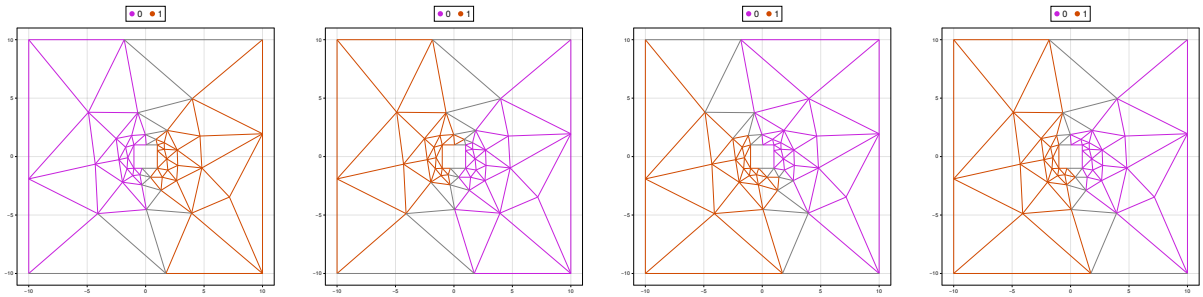
(a) Coordinate (b) Inertial (c) Spectral (d) METIS

Figure 3: Bisection partitions obtained for the `barth4` mesh.



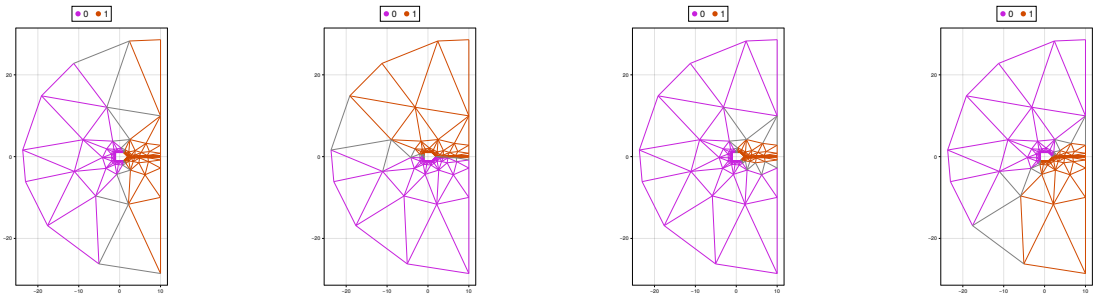
(a) Coordinate (b) Inertial (c) Spectral (d) METIS

Figure 4: Bisection partitions obtained for the `crack` mesh.



(a) Coordinate (b) Inertial (c) Spectral (d) METIS

Figure 5: Bisection partitions obtained for the `mesh1e1` mesh.



(a) Coordinate (b) Inertial (c) Spectral (d) METIS

Figure 6: Bisection partitions obtained for the `mesh2e1` mesh.

1.2. Recursive Bisection Algorithm

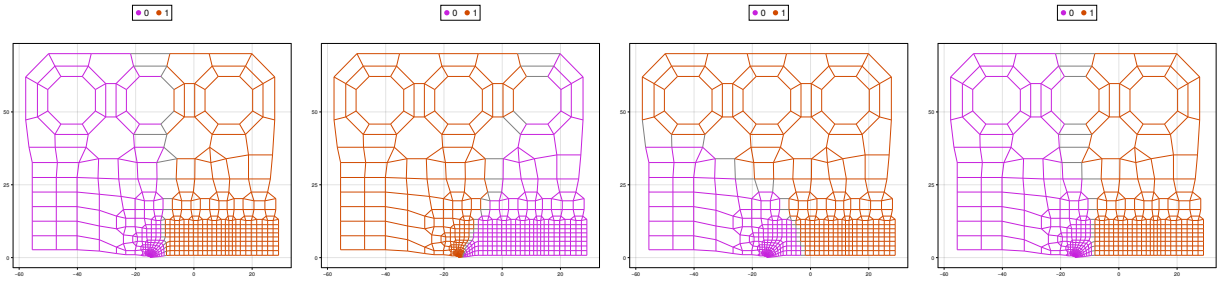
1.2.1. Implementation

Listing 3: Recursive Bisection algorithm



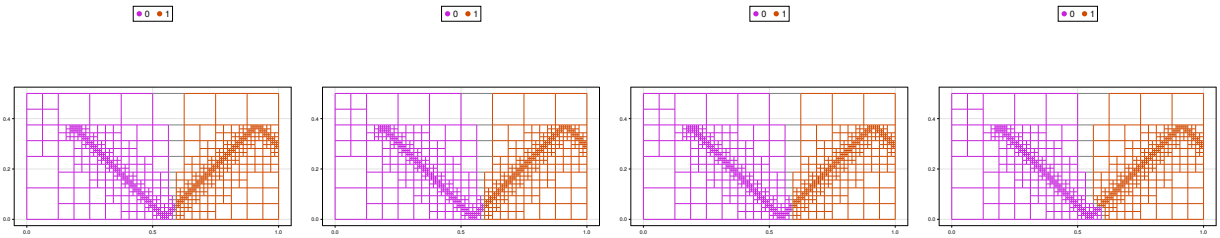
(a) Coordinate (b) Inertial (c) Spectral (d) METIS

Figure 7: Bisection partitions obtained for the `mesh3e1` mesh.



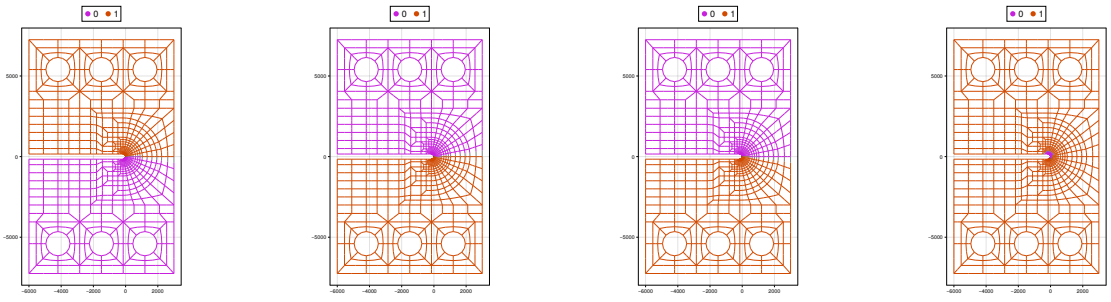
(a) Coordinate (b) Inertial (c) Spectral (d) METIS

Figure 8: Bisection partitions obtained for the `netz4504_dual` mesh.



(a) Coordinate (b) Inertial (c) Spectral (d) METIS

Figure 9: Bisection partitions obtained for the `stufe` mesh.



(a) Coordinate (b) Inertial (c) Spectral (d) METIS

Figure 10: Bisection partitions obtained for the `ukerbe1` mesh.

```
function rec_bisection(method, levels, A, coords=zeros(0), vn=zeros(0))

    n = size(A, 1)

    # Final partition vector.
```

```
    # p[i] is the final part assigned to vertex i.
    p = zeros(Int, n)

    function recursion(vertices, level, index)

        # Stop if no more bisections are required.
        if level == 0
            p[vertices] .= index
            return
        end

        # A subgraph with fewer vertices than requested final parts
        # cannot be split into 2^level non-empty parts.
        if length(vertices) < 2^level
            error("Recursive bisection failed: not enough vertices for the
                requested number of parts.")
        end

        # Extract induced subgraph.
        A_sub = A[vertices, vertices]

        # Apply chosen bisection method.
        if method == "spectral_part"
            local_p = spectral_part(A_sub)

        elseif method == "coordinate_part"
            if length(coords) == 0
                error("Coordinates are required for coordinate bisection
                    .")
            end
            coords_sub = coords[vertices, :]
            local_p = coordinate_part(A_sub, coords_sub)

        elseif method == "inertial_part"
            if length(coords) == 0
                error("Coordinates are required for inertial bisection.")
            end
            coords_sub = coords[vertices, :]
            local_p = inertial_part(A_sub, coords_sub)

        else
            error("Unknown partitioning method: $method")
        end

        # Extract the two labels returned by the bisection method.
        labels = sort(unique(local_p))

        if length(labels) != 2
            error("Bisection failed: expected 2 parts, got labels =
                $labels")
        end

        vertices_1 = vertices[findall(local_p .== labels[1])]
        vertices_2 = vertices[findall(local_p .== labels[2])]

        if isempty(vertices_1) || isempty(vertices_2)
            error("Bisection failed: one child partition is empty.")
        end

        # Each child branch contains 2^(level-1) final labels.
```

```
63          offset = 2^(level - 1)
64
65          recursion(vertices_1, level - 1, index)
66          recursion(vertices_2, level - 1, index + offset)
67
68      end
69
70      recursion(collect(1:n), levels, 1)
71
72      expected_parts = 2^levels
73      actual_parts = sort(unique(p))
74
75      if length(actual_parts) != expected_parts
76          error("Expected $expected_parts parts, got $(length(actual_parts))
                : $actual_parts")
77      end
78
79      return p
80
81  end
```

The recursive bisection routine in Listing 3 generalizes the previous two-way partitioning methods to a $p = 2^\ell$-way partition. Instead of computing all parts at once, the graph is repeatedly split into two subgraphs. After the first bisection, each of the two parts is bisected again, and this process is repeated until the prescribed number of recursion levels has been reached. Therefore, after ℓ levels, the algorithm produces 2^ℓ final subdomains.

The function takes as input the name of the bisection method, the number of recursive levels, the adjacency matrix `A`, and, when needed, the vertex coordinates `coords`. The vector `p` stores the final partition labels. Initially all entries are set to zero. The actual recursive work is performed by the inner function `recursion(vertices, level, index)`, where `vertices` contains the indices of the current subgraph, `level` is the number of remaining bisections, and `index` is the first label assigned to the current branch of the recursion tree.

The stopping criterion is reached when `level == 0`. In this case, no further bisection is required, and all vertices of the current subgraph are assigned the current final label `index`. Before performing a split, the routine also checks that the current subgraph contains at least $2^{\texttt{level}}$ vertices. Otherwise, it would be impossible to split it into the requested number of non-empty final parts. In such a case the routine raises an error, rather than silently returning a partition with fewer than 2^ℓ parts.

If further splitting is required, the induced subgraph is extracted as `A_sub = A[vertices, vertices]`. The chosen bisection method is then applied to this subgraph. For `spectral_part`, only the adjacency matrix is needed. For `coordinate_part` and `inertial_part`, the corresponding coordinates of the selected vertices are also extracted as `coords_sub`. The routine therefore uses the same local numbering inside the bisection method, but maps the result back to the original vertex indices through the array `vertices`.

After the local bisection has been computed, the algorithm extracts the two labels returned by the bisection method. The implementation explicitly checks that exactly two non-empty child partitions have been produced. If this is not the case, an error is raised. This behaviour is preferable during benchmarking, because silently assigning the whole subgraph to a single label would produce fewer than the requested 2^ℓ final parts and would make the edge-cut values misleading.

The label offset is computed as

$$\texttt{offset} = 2^{\texttt{level}-1}.$$

This is necessary because each child branch will generate $2^{\texttt{level}-1}$ final labels. The first child therefore keeps the current starting label `index`, while the second child starts at `index + offset`. For example, with $\ell = 2$, the first bisection creates two branches: the first branch eventually receives labels 1 and 2, while the second branch receives labels 3 and 4. This indexing convention guarantees that the final partition labels are consecutive and consistent with the recursion tree.

Finally, the recursion is initialized on the full vertex set `collect(1:n)` with the prescribed number of levels and initial label 1. After the recursion terminates, the implementation verifies that exactly 2^ℓ different labels have been produced. The returned vector `p` contains the final 2^ℓ-way partition of the graph.

1.2.2. Results

The recursive bisection procedure was tested by generating $p = 8$ and $p = 16$ partitions. For the recursive algorithms, this corresponds respectively to three and four levels of recursive bisection. Listing 3 shows the recursive driver used for the coordinate, inertial, and spectral bisection methods implemented above. The recursive METIS partition and the direct METIS k-way partition are instead computed using the METIS routines provided in the skeleton code. The comparison is useful because recursive bisection constructs the final partition through a sequence of two-way splits, while the k-way METIS routine computes the p-way partition directly.

As in the two-way case, the quality of the partition can be assessed through the total edge-cut, i.e. the number of graph edges connecting vertices assigned to different parts. For recursive partitioning, this quantity is especially important because each bisection step creates an interface, and the final edge-cut is the cumulative result of all these recursive splits.

The corresponding partitions are shown in Figures 11–18.

Mesh	Parts	Coordinate	Inertial	Spectral	METIS rec.	METIS k-way
`3elt`	$p = 8$	–	–	–	–	–
	$p = 16$	–	–	–	–	–
`airfoil1`	$p = 8$	–	–	–	–	–
	$p = 16$	–	–	–	–	–
`barth4`	$p = 8$	–	–	–	–	–
	$p = 16$	–	–	–	–	–
`crack`	$p = 8$	–	–	–	–	–
	$p = 16$	–	–	–	–	–
`mesh3e1`	$p = 8$	–	–	–	–	–
	$p = 16$	–	–	–	–	–
`netz4504_dual`	$p = 8$	–	–	–	–	–
	$p = 16$	–	–	–	–	–
`stufe`	$p = 8$	–	–	–	–	–
	$p = 16$	–	–	–	–	–
`ukerbe1`	$p = 8$	–	–	–	–	–
	$p = 16$	–	–	–	–	–

Table 2: Recursive partitioning edge-cuts for the considered meshes.

In the plots, small physical subdomains should not automatically be interpreted as unbalanced partitions. The load-balancing criterion is based on the number of graph vertices assigned to each part, whereas the figures show the physical geometry of the mesh. For highly non-uniform meshes, a small geometrical region can contain many vertices because of local refinement. Therefore, the edge-cut values in Table 2 should be interpreted together with the vertex counts of the parts when assessing the quality of the partition.

The first qualitative difference is visible when comparing the recursive geometrical methods with the graph-based methods. Coordinate and inertial recursive bisection tend to preserve the structure of the first geometrical cuts: the final subdomains are mostly obtained by repeatedly splitting the mesh along approximately straight directions. This gives partitions that are easy to interpret geometrically, but it may also create long interfaces when the mesh density is non-uniform.

For the `3elt` mesh, the recursive coordinate and inertial methods generate partitions that follow the global shape of the point cloud. For $p = 16$, several subdomains occupy small physical regions close to the highly refined central part of the mesh. This is expected for a non-uniform mesh: recursive bisection balances the number of graph vertices, not the physical area of the subdomains. The spectral and METIS recursive partitions are more strongly influenced by the graph connectivity, and therefore also allocate several labels close to the central refinement. These plots illustrate why geometrical area alone is not a reliable indicator of load balance for unstructured meshes; the visual inspection should be interpreted together with the part sizes and edge-cut values.

For the `airfoil1` mesh, the effect of the airfoil geometry and the refined wake region is more pronounced. The recursive coordinate and inertial partitions divide the domain into large geometric sectors. These partitions are regular and visually clear, but they do not necessarily follow the weakest graph connections. The spectral and METIS recursive partitions adapt more to the refined regions around the airfoil and the wake. In particular, several parts are placed around the leading edge, trailing edge, and wake, where the mesh is much denser. This behaviour is expected, since these regions contain many vertices and therefore require more subdomains if the partition is to remain balanced in terms of graph vertices rather than physical area.

For the `barth4` and `crack` meshes, the same qualitative behaviour can be observed, but the effect of the geometry is different. The `barth4` mesh has a strongly non-uniform vertex distribution, with many vertices concentrated around the central region. As a consequence, some recursive partitions appear to occupy a very small physical area near the centre of the mesh. This does not necessarily imply that the partition is unbalanced in terms of graph vertices, since the central region is much more densely discretized than the outer part of the domain. Nevertheless, these plots show a limitation of interpreting graph partitions only from their geometrical area.

The `crack` mesh is instead characterized by a localized internal feature around the centre. The recursive coordinate and inertial methods generate partitions that remain mostly aligned with successive geometrical cuts. For $p = 8$ and $p = 16$, this creates large block-like regions away from the crack and much smaller regions around the refined centre. The spectral and METIS-based methods are more sensitive to the graph connectivity near the crack and tend to isolate the refined central region more strongly. This is expected, because the high local mesh density around the crack contains many graph vertices and therefore receives several subdomains when the partition is balanced by vertex count.

The `mesh3e1` mesh is useful as a reference case because it is much more regular than the previous unstructured examples. In this case, the recursive coordinate and inertial methods produce partitions that are mostly block-like and aligned with the geometry of the square domain. Since the mesh is close to structured, these geometrical cuts are already quite reasonable. The spectral and METIS-based methods also produce compact subdomains, but their interfaces are less constrained by coordinate directions and can adapt to the graph connectivity. For this mesh, the visual difference between the methods is less dramatic than for highly non-uniform meshes such as `airfoil1`, `barth4`, or `crack`.

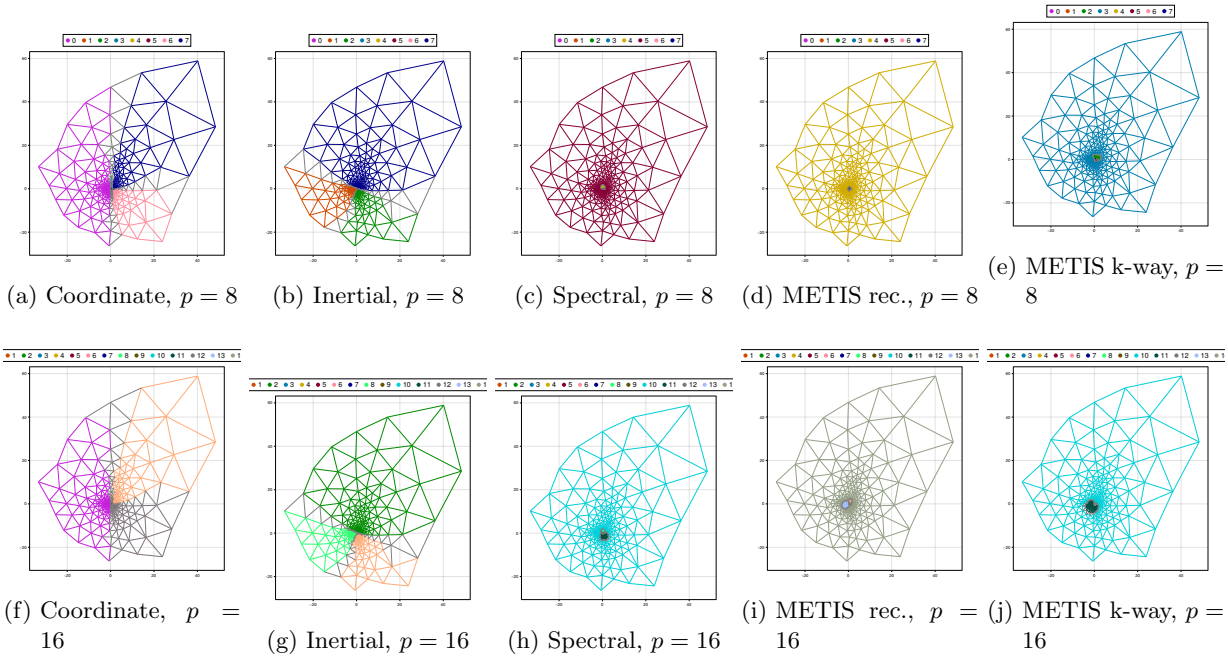
The `netz4504_dual` mesh has a more complex geometry, with large coarse cells in the upper part and a much finer structured region near the bottom boundary. This makes the recursive partitioning behaviour particularly interesting. The coordinate and inertial methods tend to split the domain according to its global geometric extent, producing long vertical or horizontal regions. By contrast, the spectral and METIS partitions allocate several smaller parts close to the refined bottom region. This is consistent with the fact that graph partitioning balances the number of vertices rather than the physical area of the subdomains. Therefore, regions that are geometrically small but highly discretized may receive several partition labels.

The `stufe` mesh contains a step-like geometry, with two refined diagonal regions and a relatively coarse outer rectangular structure. The recursive coordinate and inertial methods again produce partitions that are strongly driven by the global geometry. As a consequence, the parts tend to form vertical or horizontal blocks, with smaller subdomains appearing around the refined regions. The graph-based methods produce more irregular partitions, especially for $p = 16$, because the separator is allowed to follow the local mesh connectivity rather than a global coordinate direction. This gives a less visually regular decomposition, but it can reduce the number of edges crossing between parts.

The `ukerbe1` mesh is one of the most challenging examples, because it combines a large structured geometry, several circular holes, and a pronounced notch-like feature near the centre. For this mesh, the recursive coordinate and inertial methods split the domain mostly into large upper and lower or left and right regions, with additional refinement around the notch in later levels of the recursion. The spectral and METIS-based methods instead place several separators close to the central notch, which acts as a natural bottleneck of the graph. This is a reasonable behaviour for graph partitioning: cutting through a narrow, highly connected transition region can reduce the final edge-cut. However, as in the previous non-uniform meshes, some partitions appear to occupy a small physical area. This should not immediately be interpreted as poor load balance, since the local mesh density is much higher near the notch.

The comparison with METIS k-way partitioning is useful because it shows the difference between recursively applying a two-way partitioner and computing the final p-way partition directly. The k-way METIS

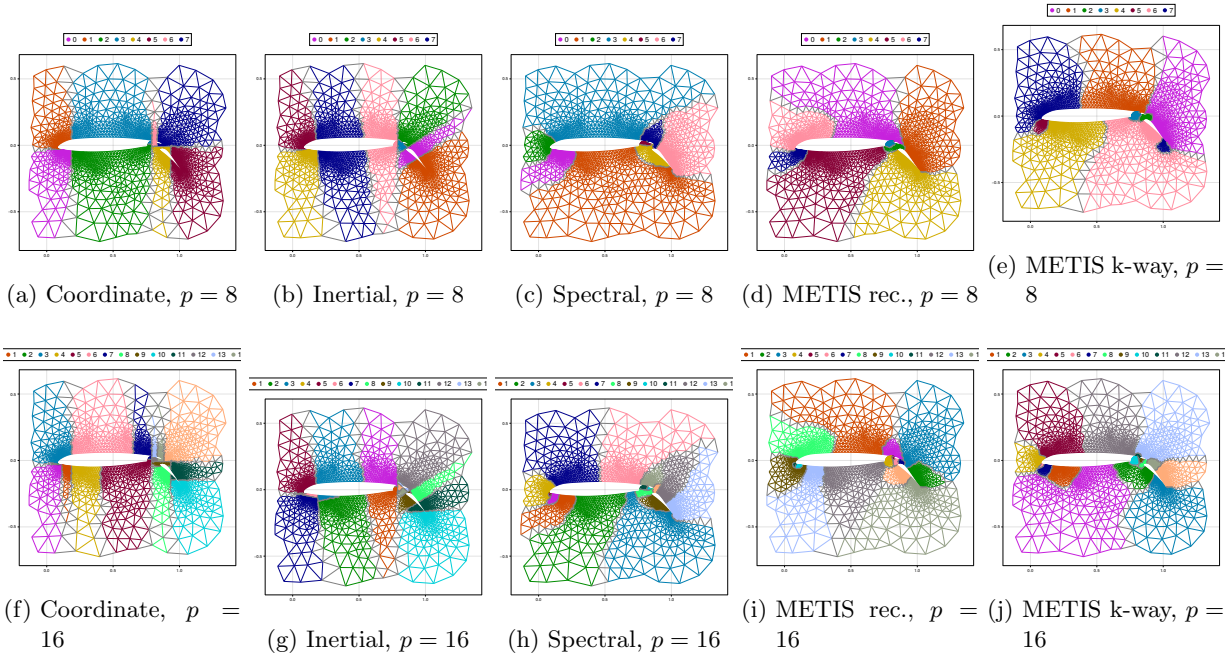
partitions are generally more globally optimized: the algorithm can account for all p parts simultaneously, rather than committing to early bisection decisions that cannot be changed later. Therefore, one expects METIS k-way partitioning to give lower edge-cuts than recursive partitioning, especially for $p = 16$, where early suboptimal splits are amplified by the additional recursion levels.



(a) Coordinate, $p = 8$ (b) Inertial, $p = 8$ (c) Spectral, $p = 8$ (d) METIS rec., $p = 8$ (e) METIS k-way, $p = 8$

(f) Coordinate, $p = 16$ (g) Inertial, $p = 16$ (h) Spectral, $p = 16$ (i) METIS rec., $p = 16$ (j) METIS k-way, $p = 16$

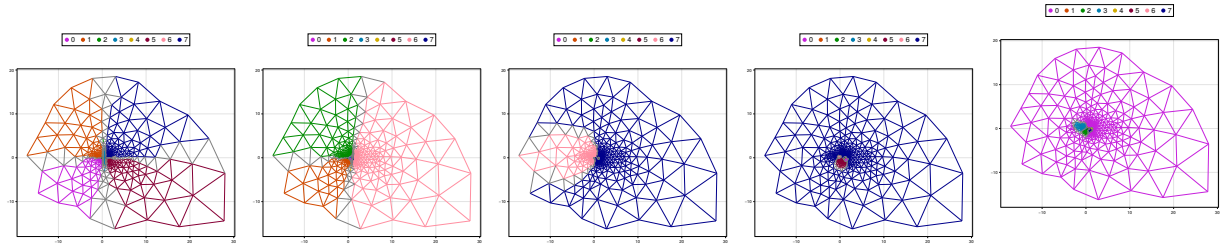
Figure 11: Recursive and k-way partitions obtained for the `3elt` mesh with $p = 8$ and $p = 16$ parts.



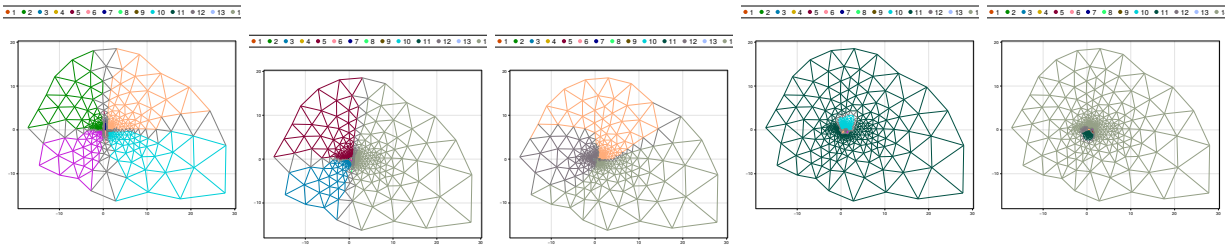
(a) Coordinate, $p = 8$ (b) Inertial, $p = 8$ (c) Spectral, $p = 8$ (d) METIS rec., $p = 8$ (e) METIS k-way, $p = 8$

(f) Coordinate, $p = 16$ (g) Inertial, $p = 16$ (h) Spectral, $p = 16$ (i) METIS rec., $p = 16$ (j) METIS k-way, $p = 16$

Figure 12: Recursive and k-way partitions obtained for the `airfoil1` mesh with $p = 8$ and $p = 16$ parts.

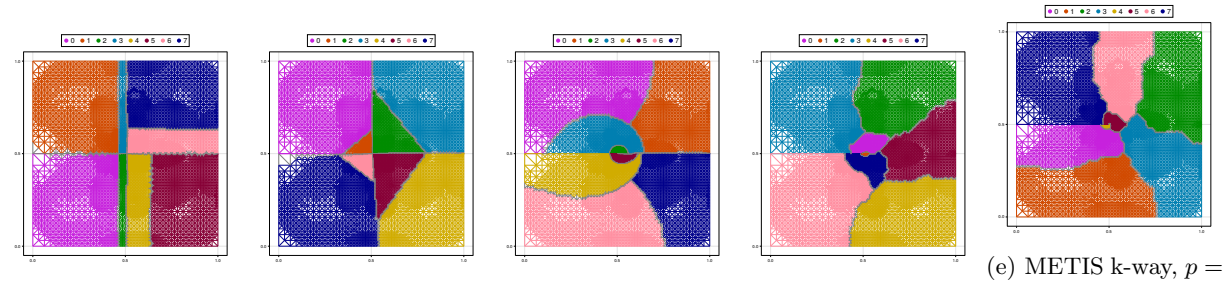


(a) Coordinate, $p = 8$ (b) Inertial, $p = 8$ (c) Spectral, $p = 8$ (d) METIS rec., $p = 8$ (e) METIS k-way, $p = 8$

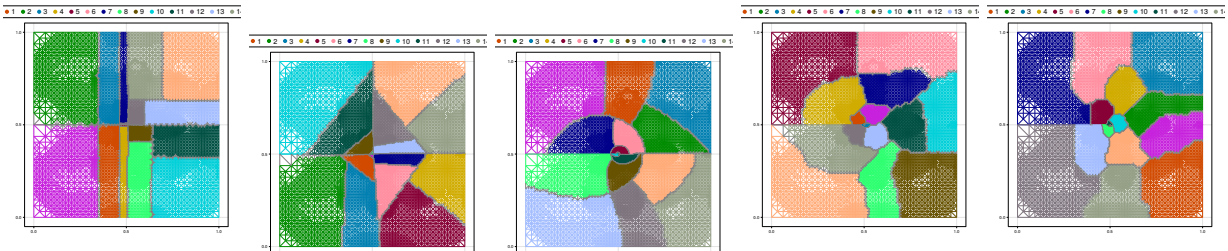


(f) Coordinate, $p = 16$ (g) Inertial, $p = 16$ (h) Spectral, $p = 16$ (i) METIS rec., $p = 16$ (j) METIS k-way, $p = 16$

Figure 13: Recursive and k-way partitions obtained for the `barth4` mesh with $p = 8$ and $p = 16$ parts.

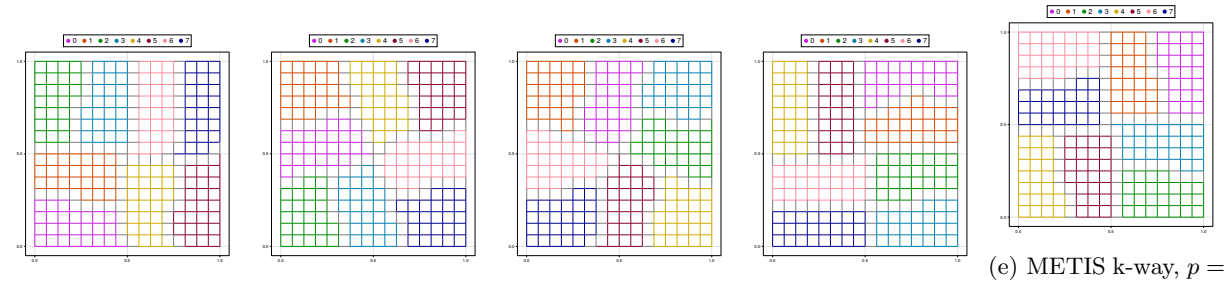


(a) Coordinate, $p = 8$ (b) Inertial, $p = 8$ (c) Spectral, $p = 8$ (d) METIS rec., $p = 8$ (e) METIS k-way, $p = 8$

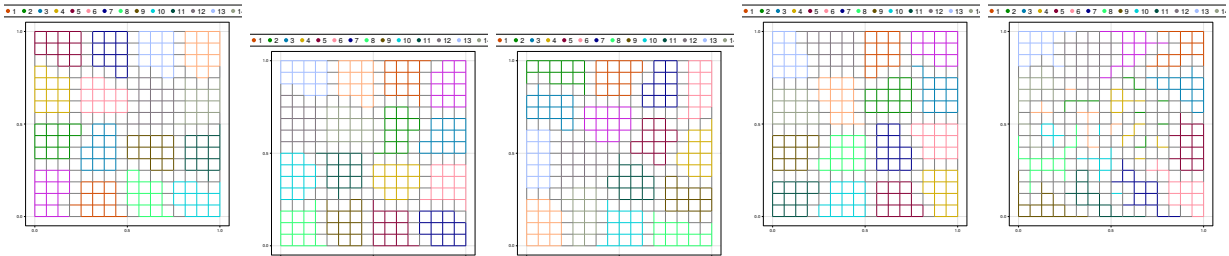


(f) Coordinate, $p = 16$ (g) Inertial, $p = 16$ (h) Spectral, $p = 16$ (i) METIS rec., $p = 16$ (j) METIS k-way, $p = 16$

Figure 14: Recursive and k-way partitions obtained for the `crack` mesh with $p = 8$ and $p = 16$ parts.

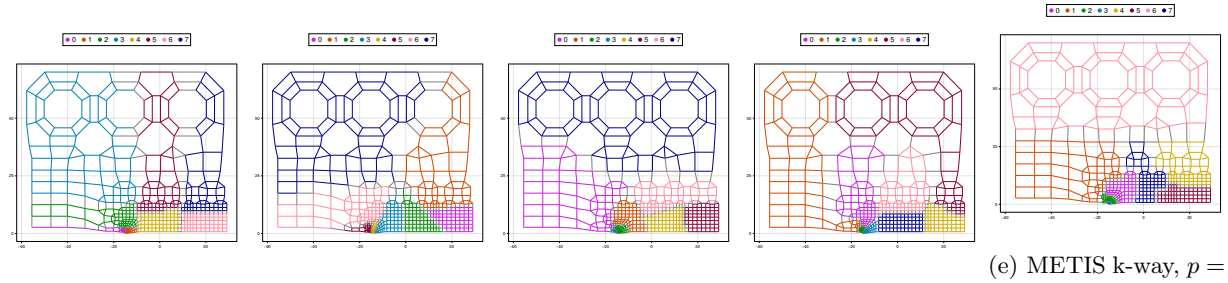


(a) Coordinate, $p = 8$ (b) Inertial, $p = 8$ (c) Spectral, $p = 8$ (d) METIS rec., $p = 8$ (e) METIS k-way, $p = 8$

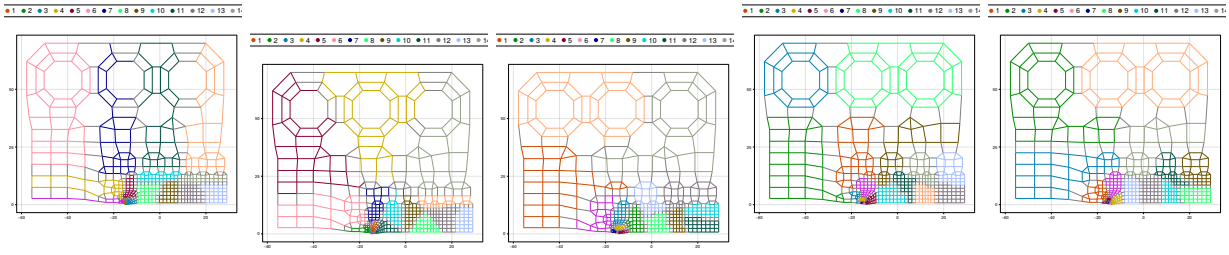


(f) Coordinate, $p = 16$ (g) Inertial, $p = 16$ (h) Spectral, $p = 16$ (i) METIS rec., $p = 16$ (j) METIS k-way, $p = 16$

Figure 15: Recursive and k-way partitions obtained for the `mesh3e1` mesh with $p = 8$ and $p = 16$ parts.

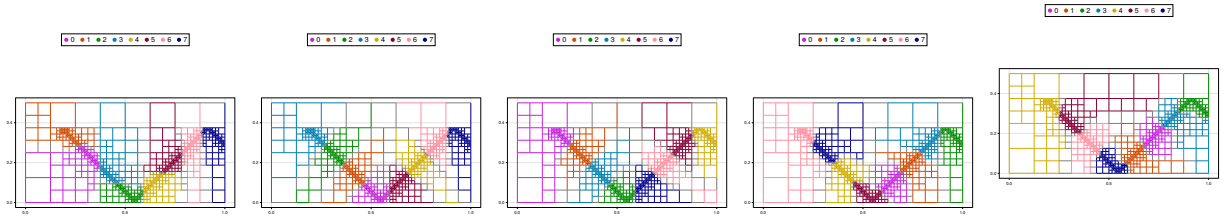


(a) Coordinate, $p = 8$ (b) Inertial, $p = 8$ (c) Spectral, $p = 8$ (d) METIS rec., $p = 8$ (e) METIS k-way, $p = 8$

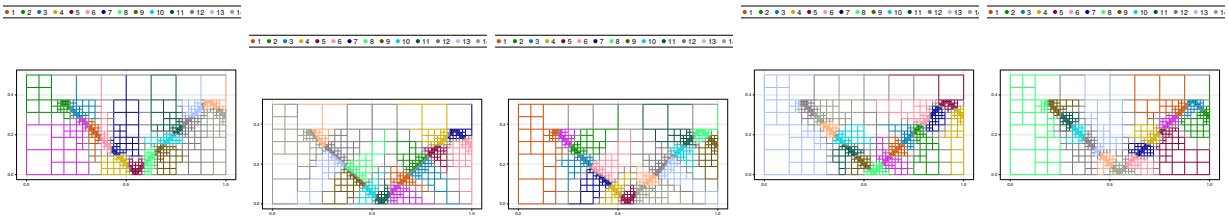


(f) Coordinate, $p = 16$ (g) Inertial, $p = 16$ (h) Spectral, $p = 16$ (i) METIS rec., $p = 16$ (j) METIS k-way, $p = 16$

Figure 16: Recursive and k-way partitions obtained for the `netz4504_dual` mesh with $p = 8$ and $p = 16$ parts.

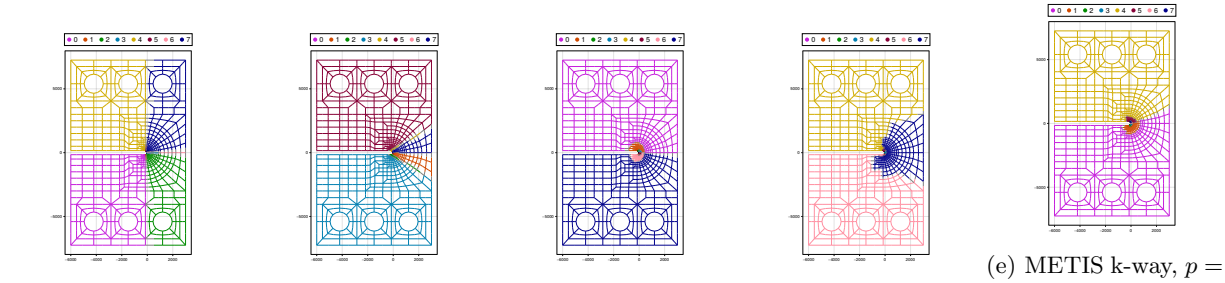


(a) Coordinate, $p = 8$ (b) Inertial, $p = 8$ (c) Spectral, $p = 8$ (d) METIS rec., $p = 8$ (e) METIS k-way, $p = 8$

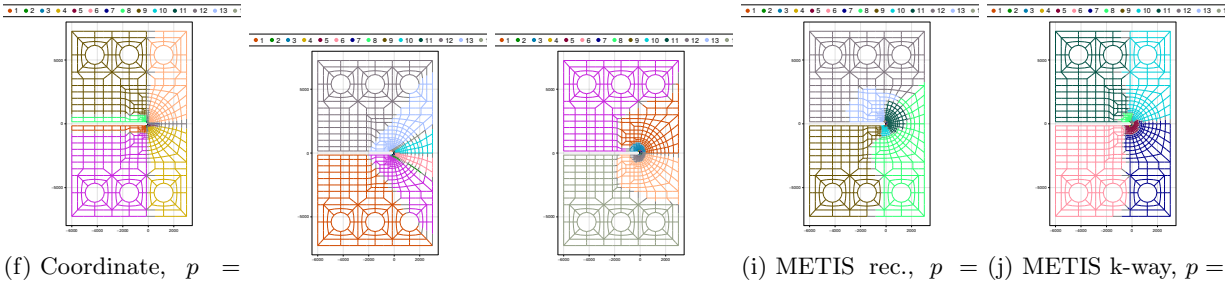


(f) Coordinate, $p = 16$ (g) Inertial, $p = 16$ (h) Spectral, $p = 16$ (i) METIS rec., $p = 16$ (j) METIS k-way, $p = 16$

Figure 17: Recursive and k-way partitions obtained for the `stufe` mesh with $p = 8$ and $p = 16$ parts.



(a) Coordinate, $p = 8$ (b) Inertial, $p = 8$ (c) Spectral, $p = 8$ (d) METIS rec., $p = 8$ (e) METIS k-way, $p = 8$



(f) Coordinate, $p = 16$ (g) Inertial, $p = 16$ (h) Spectral, $p = 16$ (i) METIS rec., $p = 16$ (j) METIS k-way, $p = 16$

Figure 18: Recursive and k-way partitions obtained for the `ukerbe1` mesh with $p = 8$ and $p = 16$ parts.

2. HPC software frameworks [50 points]

2.1. Mathematical Formulation

In this section we solve the two-dimensional Poisson problem with homogeneous Dirichlet boundary conditions,

$$\begin{aligned} -\Delta u &= f && \text{in } \Omega, \\ u &= 0 && \text{on } \partial\Omega, \end{aligned}$$

where the computational domain is the unit square

$$\Omega = (0,1)^2,$$

and the source term is constant,

$$f(x_1, x_2) = 20.$$

In Cartesian coordinates the Laplace operator is

$$\Delta u = \frac{\partial^2 u}{\partial x_1^2} + \frac{\partial^2 u}{\partial x_2^2}.$$

The homogeneous Dirichlet condition imposes that the solution is zero on the whole boundary of the square. Therefore, the expected qualitative behaviour is a positive solution in the interior, symmetric with respect to the two midlines of the square, and attaining its maximum close to the centre of the domain.

The domain is discretized by a uniform Cartesian grid with m_x grid points in the x_1-direction and m_y grid points in the x_2-direction, including boundary points. The mesh widths are

$$h_x = \frac{1}{m_x - 1}, \qquad h_y = \frac{1}{m_y - 1}.$$

The grid points are

$$x_i = i h_x, \qquad i = 0, \ldots, m_x - 1,$$

and

$$y_j = j h_y, \qquad j = 0, \ldots, m_y - 1.$$

The numerical approximation of $u(x_i, y_j)$ is denoted by

$$u_{i,j} \approx u(x_i, y_j).$$

For an interior grid point, namely for

$$1 \le i \le m_x - 2, \qquad 1 \le j \le m_y - 2,$$

the second derivatives are approximated with second-order centred finite differences:

$$\frac{\partial^2 u}{\partial x_1^2}(x_i, y_j) = \frac{u_{i-1,j} - 2u_{i,j} + u_{i+1,j}}{h_x^2} + \mathcal{O}(h_x^2),$$

and

$$\frac{\partial^2 u}{\partial x_2^2}(x_i, y_j) = \frac{u_{i,j-1} - 2u_{i,j} + u_{i,j+1}}{h_y^2} + \mathcal{O}(h_y^2).$$

Substituting these approximations into $-\Delta u = 20$ gives

$$-\frac{u_{i-1,j} - 2u_{i,j} + u_{i+1,j}}{h_x^2} - \frac{u_{i,j-1} - 2u_{i,j} + u_{i,j+1}}{h_y^2} = 20.$$

Equivalently,

$$\left(\frac{2}{h_x^2} + \frac{2}{h_y^2}\right) u_{i,j} - \frac{1}{h_x^2}\left(u_{i-1,j} + u_{i+1,j}\right) - \frac{1}{h_y^2}\left(u_{i,j-1} + u_{i,j+1}\right) = 20.$$

This is the standard five-point finite-difference stencil for the negative Laplacian.

In the implementation, the equation is multiplied by $h_x h_y$. This does not change the solution of the linear system, but leads to the coefficients used in the assembled PETSc matrix:

$$2\left(\frac{h_x}{h_y} + \frac{h_y}{h_x}\right) u_{i,j} - \frac{h_y}{h_x}\left(u_{i-1,j} + u_{i+1,j}\right) - \frac{h_x}{h_y}\left(u_{i,j-1} + u_{i,j+1}\right) = 20 h_x h_y. \tag{1}$$

Therefore, the matrix stencil used for an interior point is

$$\begin{matrix} & -h_x/h_y & \\ -h_y/h_x & 2(h_x/h_y + h_y/h_x) & -h_y/h_x \\ & -h_x/h_y & \end{matrix} \tag{2}$$

where the horizontal neighbours correspond to the coefficients $-h_y/h_x$, while the vertical neighbours correspond to the coefficients $-h_x/h_y$.

The boundary condition is imposed strongly. If a node lies on the boundary, that is if

$$i = 0, \quad i = m_x - 1, \quad j = 0, \quad \text{or} \quad j = m_y - 1,$$

then the discrete equation is replaced by

$$u_{i,j} = 0.$$

In matrix form, this corresponds to inserting an identity row for every boundary degree of freedom:

$$A_{kk} = 1, \qquad b_k = 0.$$

For interior degrees of freedom, the finite-difference equations define a sparse linear system

$$A\mathbf{u} = \mathbf{b},$$

where $\mathbf{u}$ contains the nodal values $u_{i,j}$. If the interior unknowns are ordered lexicographically, for example with index

$$k = i + m_x j,$$

then each row of A contains at most five nonzero entries, corresponding to the centre point and its four nearest neighbours. Since the operator is the negative Laplacian with homogeneous Dirichlet boundary conditions, the resulting matrix is symmetric positive definite on the constrained problem. This property motivates the use of Krylov methods for symmetric positive definite systems, such as the conjugate-gradient method.

2.2. PETSc Implementation

The Poisson problem was implemented in C using PETSc. The code uses the PETSc distributed array abstraction `DMDA`, which is particularly convenient for finite-difference discretizations on structured grids. The distributed array stores the two-dimensional Cartesian layout of the grid and distributes grid points among MPI processes. In this way, each process assembles only the rows of the matrix and the entries of the right-hand side vector that belong to its local portion of the grid.

The main PETSc objects used in the code are:

`DM`, `KSP`, `Mat`, `Vec`.

The `DM` object describes the distributed structured grid, while the `KSP` object manages the linear solver. The matrix and the right-hand side are assembled through callback functions registered with the `KSP` object.

Listing 4: Creation of the PETSc solver and distributed grid

```
1  PetscCall(PetscInitialize(&argc, &argv, NULL, help));
2
3  PetscCall(KSPCreate(PETSC_COMM_WORLD, &ksp));
4
5  PetscCall(DMDACreate2d(
6      PETSC_COMM_WORLD,
7      DM_BOUNDARY_NONE,
8      DM_BOUNDARY_NONE,
9      DMDA_STENCIL_STAR,
10     3, 3,
11     PETSC_DECIDE, PETSC_DECIDE,
12     1, 1,
13     NULL, NULL,
14     &da));
15
```

```
PetscCall(DMSetFromOptions(da));
PetscCall(DMSetUp(da));

PetscCall(DMDASetUniformCoordinates(da, 0.0, 1.0, 0.0, 1.0, 0.0, 0.0));
PetscCall(DMDASetFieldName(da, 0, "Solution"));

PetscCall(KSPSetDM(ksp, da));
PetscCall(KSPSetComputeRHS(ksp, ComputeRHS, NULL));
PetscCall(KSPSetComputeOperators(ksp, ComputeMatrix, NULL));

PetscCall(KSPSetFromOptions(ksp));
PetscCall(KSPSetUp(ksp));

PetscCall(KSPSolve(ksp, NULL, NULL));
```

The call to `DMDACreate2d` creates a two-dimensional distributed array. The options

`DM_BOUNDARY_NONE`

are used because the problem does not have periodic boundary conditions. The option

`DMDA_STENCIL_STAR`

corresponds to the five-point stencil of the finite-difference discretization: each interior point couples only to itself and to its north, south, east, and west neighbours. The stencil width is set to one, consistently with the second-order centred discretization.

The grid is initialized with default size 3×3, but this value can be overridden from the command line by PETSc options such as

Listing 5: Example of runtime grid-size selection

```
-da_grid_x 512 -da_grid_y 512
```

This is useful because it allows the same executable to be used for several problem resolutions without recompilation.

The right-hand side vector is assembled in the function `ComputeRHS`. Each MPI process first obtains the portion of the grid that it owns through `DMDAGetCorners`. Then it writes the corresponding local entries of the right-hand side vector. Boundary entries are set to zero, while interior entries are set to $20h_xh_y$, consistently with the scaled finite-difference equation (1).

Listing 6: Assembly of the right-hand side vector

```
PetscCall(DMDAGetInfo(da, NULL, &mx, &my, NULL,
                      NULL, NULL, NULL, NULL,
                      NULL, NULL, NULL, NULL, NULL));

Hx = 1.0 / (PetscReal)(mx - 1);
Hy = 1.0 / (PetscReal)(my - 1);

PetscCall(DMDAGetCorners(da, &xs, &ys, NULL, &xm, &ym, NULL));
PetscCall(DMDAVecGetArray(da, b, &array));

for (j = ys; j < ys + ym; j++) {
  for (i = xs; i < xs + xm; i++) {

    if (i == 0 || j == 0 || i == mx - 1 || j == my - 1) {
      array[j][i] = 0.0;
    } else {
      array[j][i] = 20.0 * Hx * Hy;
    }

  }
}

```

```
PetscCall(DMDAVecRestoreArray(da, b, &array));
PetscCall(VecAssemblyBegin(b));
PetscCall(VecAssemblyEnd(b));
```

The matrix is assembled in the function `ComputeMatrix`. Boundary nodes are treated by inserting an identity row. For interior nodes, the five-point stencil (2) is inserted using `MatSetValuesStencil`. Using stencil indices avoids manually converting the two-dimensional grid indices (i, j) into global algebraic indices.

Listing 7: Assembly of the finite-difference matrix

```
Hx = 1.0 / (PetscReal)(mx - 1);
Hy = 1.0 / (PetscReal)(my - 1);

HxdHy = Hx / Hy;
HydHx = Hy / Hx;

PetscCall(DMDAGetCorners(da, &xs, &ys, NULL, &xm, &ym, NULL));

for (j = ys; j < ys + ym; j++) {
  for (i = xs; i < xs + xm; i++) {

    row.i = i;
    row.j = j;

    if (i == 0 || j == 0 || i == mx - 1 || j == my - 1) {

      v[0] = 1.0;
      PetscCall(MatSetValuesStencil(jac, 1, &row, 1, &row,
                                    v, INSERT_VALUES));

    } else {

      PetscInt ncols = 0;

      if (j - 1 > 0) {
        v[ncols]     = -HxdHy;
        col[ncols].i = i;
        col[ncols].j = j - 1;
        ncols++;
      }

      if (i - 1 > 0) {
        v[ncols]     = -HydHx;
        col[ncols].i = i - 1;
        col[ncols].j = j;
        ncols++;
      }

      v[ncols]     = 2.0 * (HxdHy + HydHx);
      col[ncols].i = i;
      col[ncols].j = j;
      ncols++;

      if (i + 1 < mx - 1) {
        v[ncols]     = -HydHx;
        col[ncols].i = i + 1;
        col[ncols].j = j;
        ncols++;
      }

      if (j + 1 < my - 1) {
        v[ncols]     = -HxdHy;
```

```
53          col[ncols].i = i;
54          col[ncols].j = j + 1;
55          ncols++;
56        }
57
58        PetscCall(MatSetValuesStencil(jac, 1, &row, ncols, col,
59                                      v, INSERT_VALUES));
60      }
61    }
62 }
63
64 PetscCall(MatAssemblyBegin(jac, MAT_FINAL_ASSEMBLY));
65 PetscCall(MatAssemblyEnd(jac, MAT_FINAL_ASSEMBLY));
66
67 PetscCall(MatSetOption(jac, MAT_SYMMETRIC, PETSC_TRUE));
```

Boundary neighbours are not inserted in the matrix for interior rows. This is consistent with the homogeneous Dirichlet condition, because boundary values are known and equal to zero. If the boundary condition were non-homogeneous, the contribution of neighbouring boundary values would instead have to be moved to the right-hand side.

The implementation does not hard-code a specific Krylov method or preconditioner. Instead, it calls

`KSPSetFromOptions(ksp)`,

which allows the solver configuration to be chosen at runtime from the PETSc options database. Since the assembled operator is symmetric positive definite, a natural choice is the conjugate-gradient method. In the runs reported here, the linear system was solved with a Krylov method selected through PETSc options, for example

Listing 8: Example PETSc solver options for the Poisson problem

```
1 -ksp_type cg -pc_type jacobi -ksp_rtol 1e-10 -ksp_monitor
```

For larger grids, a multigrid preconditioner such as PETSc's algebraic multigrid preconditioner can also be used by changing only the runtime options, for example

Listing 9: Example PETSc options using algebraic multigrid

```
1 -ksp_type cg -pc_type gamg -ksp_rtol 1e-10 -ksp_monitor
```

This separation between the discretization code and the solver configuration is one of the main advantages of PETSc: the same application code can be tested with different iterative methods, tolerances, and preconditioners without modifying or recompiling the source code.

After solving the system, the numerical solution is written to an ASCII file containing the coordinates and the corresponding solution value. Since the solution vector is distributed across MPI processes, the output is performed using synchronized PETSc ASCII printing.

Listing 10: Writing the distributed solution to file

```
1  PetscCall(PetscViewerASCIIOpen(PETSC_COMM_WORLD, filename, &viewer));
2  PetscCall(PetscViewerASCIIPrintf(viewer, "# x y u\n"));
3
4  PetscCall(DMDAVecGetArrayRead(da, x, &array));
5  PetscCall(PetscViewerASCIIPushSynchronized(viewer));
6
7  for (j = ys; j < ys + ym; j++) {
8    for (i = xs; i < xs + xm; i++) {
9      PetscReal xcoord = ((PetscReal)i) * Hx;
10     PetscReal ycoord = ((PetscReal)j) * Hy;
11     PetscReal value  = PetscRealPart(array[j][i]);
12
13     PetscCall(PetscViewerASCIISynchronizedPrintf(
14         viewer,
15         "%.16e %.16e %.16e\n",
16         (double)xcoord,
```

```
            (double)ycoord,
            (double)value));
    }
}

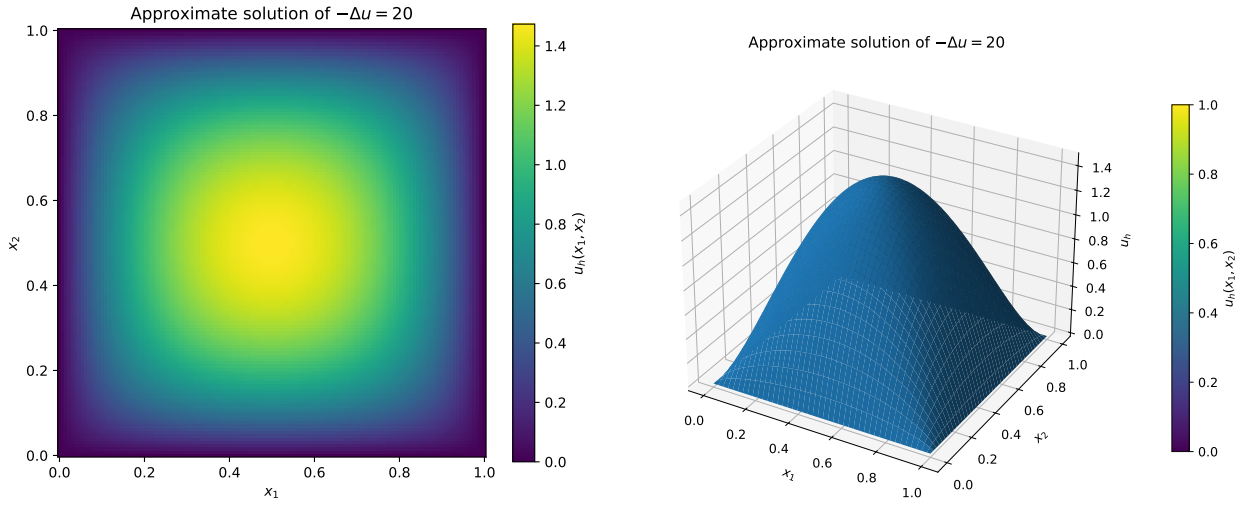
PetscCall(PetscViewerFlush(viewer));
PetscCall(PetscViewerASCIIPopSynchronized(viewer));
PetscCall(DMDAVecRestoreArrayRead(da, x, &array));
PetscCall(PetscViewerDestroy(&viewer));
```

2.3. Visualization

The computed solution was exported as a file containing triples

$$(x_i, y_j, u_{i,j}),$$

which were then postprocessed in Python to generate a heatmap and a three- dimensional surface plot. These visualizations are shown in Figure 19.



(a) Heatmap of the numerical solution.

(b) Surface plot of the numerical solution.

Figure 19: Approximate solution of the Poisson problem $-\Delta u = 20$ in $\Omega = (0,1)^2$, with homogeneous Dirichlet boundary conditions on $\partial\Omega$. The solution is zero on the boundary and positive in the interior, with a maximum close to the centre of the domain, as expected from the symmetry of the problem and the positivity of the source term.

The plots confirm the expected qualitative structure of the solution. Because the forcing term is positive and the boundary values are zero, the discrete solution is positive inside the domain and vanishes on the boundary. Moreover, since both the geometry and the source term are symmetric with respect to the lines $x_1 = 1/2$ and $x_2 = 1/2$, the numerical solution is also symmetric. The maximum is attained near the centre of the square, while the solution decreases smoothly towards zero when approaching the boundary. This behaviour is consistent with the maximum principle for the Poisson equation and provides a qualitative validation of the finite-difference discretization and PETSc implementation.

2.4. Strong Scaling Analysis

The final part of the PETSc study consists of a strong scaling experiment for the two-dimensional Poisson solver.
For a fixed grid resolution $n \times n$, let

$$T_p$$

denote the median runtime obtained with p MPI processes. The strong scaling speedup is defined as

$$S_p = \frac{T_1}{T_p},$$

where T_1 is the runtime of the corresponding serial run. The ideal strong scaling behaviour is

$$S_p^{\text{ideal}} = p,$$

which is equivalent to

$$T_p^{\text{ideal}} = \frac{T_1}{p}.$$

The corresponding parallel efficiency is

$$E_p = \frac{S_p}{p} = \frac{T_1}{pT_p}.$$

In the plots, this quantity is reported in percent as $100E_p$. A value close to 100% means that doubling the number of processes almost halves the runtime, while a lower value indicates that communication, synchronization, and parallel overheads are becoming significant compared with the local computation.

The strong scaling experiment was performed on the Euler cluster using one MPI process per node. This choice makes the scaling study a genuine multi-node MPI experiment and avoids mixing the effects of intra-node and inter-node parallelism.

Listing 11: SLURM configuration used for the strong scaling study

```
#SBATCH --job-name=poisson_strong_petsc
#SBATCH --nodes=32
#SBATCH --ntasks-per-node=1
#SBATCH --cpus-per-task=1
#SBATCH --time=02:00:00
#SBATCH --constraint=EPYC_7763

module purge
module load stack/2025-06
module load gcc/12.2.0
module load openmpi/5.0.5
module load petsc/3.22.1
```

The benchmark was run for the two representative grid resolutions

$$n = 512, \qquad n = 1024,$$

and for

$$p = 1, 2, 4, 8, 16, 32$$

MPI processes. Although the benchmark script was initially prepared to also allow a run with $n = 2048$, this case was not included in the final strong scaling campaign and is therefore not shown in the plots. The final discussion is thus based only on the completed and plotted measurements for $n = 512$ and $n = 1024$.

For each pair (n, p), three repetitions were performed and the median value was used in the plots. This reduces the influence of occasional runtime fluctuations caused by system noise, network contention, or temporary load variations on the cluster.

Listing 12: Representative benchmark loop for the strong scaling experiment

```
RESOLUTIONS=(512 1024)
PROCS=(1 2 4 8 16 32)
REPS=3

PETSC_OPTS="-ksp_type cg -pc_type gamg -ksp_rtol 1e-8 -
    ksp_converged_reason"

for n in "${RESOLUTIONS[@]}"; do
  for p in "${PROCS[@]}"; do
    for rep in $(seq 1 "$REPS"); do

      mpirun \
        --hostfile "$HOSTFILE" \
        -np "$p" \
        --map-by ppr:1:node \
```

```
        --bind-to core \
        ./poisson2d_benchmark \
        -da_grid_x "$n" -da_grid_y "$n" \
        $PETSC_OPTS

    done
  done
done
```

The PETSc linear solver was configured at runtime as

`-ksp_type cg -pc_type gamg -ksp_rtol 1e-8`.

This means that the linear system was solved using the conjugate-gradient method, preconditioned by PETSc's algebraic multigrid preconditioner `GAMG`. This is a suitable choice for the present problem because the finite-difference discretization of the negative Laplacian with homogeneous Dirichlet boundary conditions gives a symmetric positive definite linear system. Moreover, multigrid-type preconditioners are well suited for elliptic operators such as the Poisson operator, since they are designed to reduce errors over several length scales efficiently.

Figure 20 shows the median time to solution as a function of the number of MPI processes. The runtime decreases when increasing the process count, showing that the PETSc implementation is able to exploit parallelism. However, the scaling is not ideal over the full range of process counts. This is expected in a strong scaling setting: as p increases, the local subdomain assigned to each MPI rank becomes smaller, and the relative cost of communication and synchronization increases.

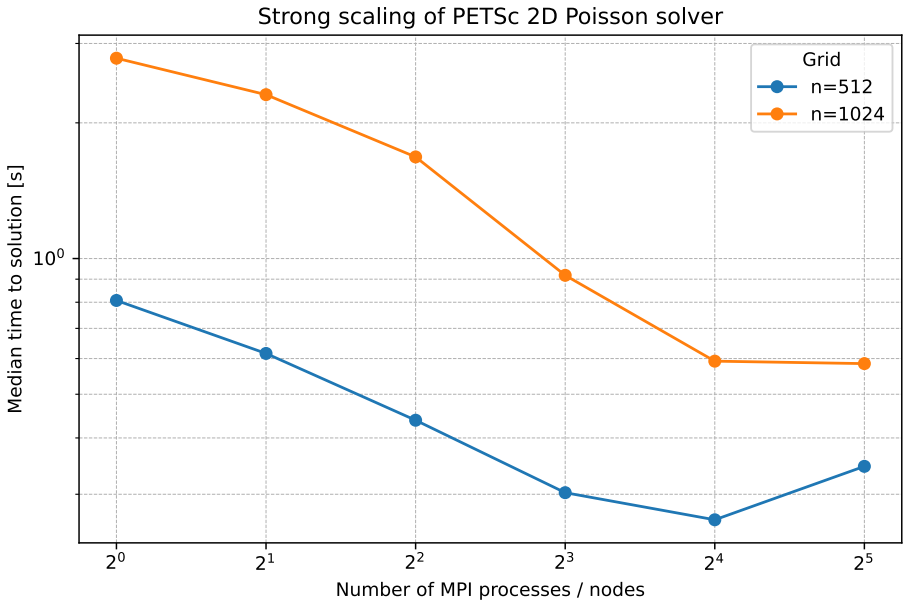


Figure 20: Strong scaling of the PETSc two-dimensional Poisson solver. The plot reports the median total time to solution for the grid resolutions $n = 512$ and $n = 1024$. The global problem size is fixed while the number of MPI processes, and therefore the number of nodes, is increased.

A useful way to interpret the result is to distinguish the total time from the linear-solve time. The total time includes not only the Krylov solve, but also the PETSc setup, creation of the distributed grid, matrix and vector assembly, and other initialization costs. By contrast, the solve time isolates the `KSPSolve` phase, which is the most relevant part for assessing the parallel performance of the iterative linear solver itself.

Figure 21 shows the strong scaling behaviour of the `KSPSolve` phase. The solve time decreases more clearly than the total time, especially for the larger resolution. This confirms that the parallelization of the algebraic solve is effective. The larger grid $n = 1024$ provides more work per MPI process than the smaller grid $n = 512$, and therefore has a better computation-to-communication ratio. As a result, it is expected to scale better to larger process counts.

The difference between the two resolutions is also visible in the parallel efficiency, shown in Figure 22. For small numbers of processes, the efficiency remains relatively high, meaning that the additional nodes are

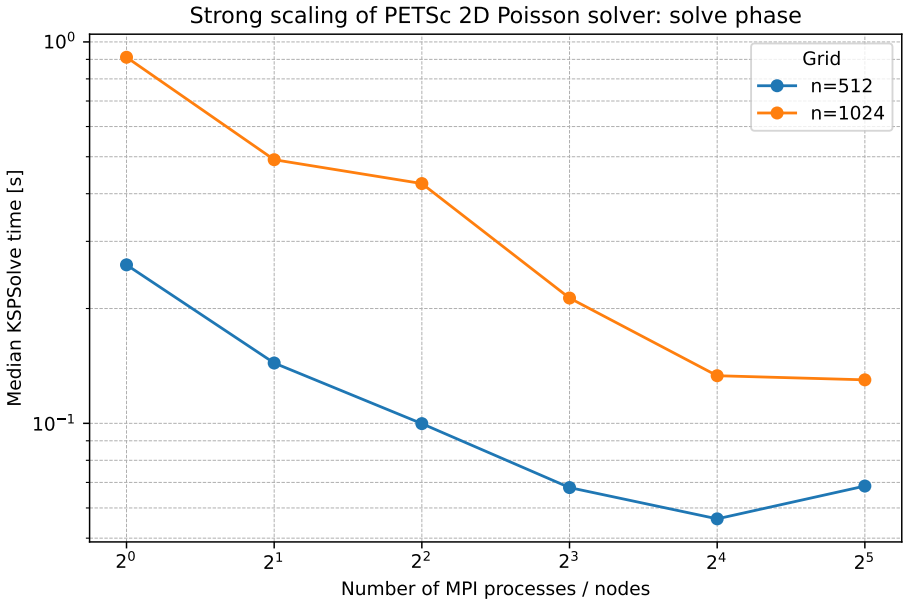


Figure 21: Strong scaling of the PETSc solver phase. The plot reports the median time spent inside `KSPSolve`. Compared with the total runtime, this isolates the performance of the Krylov solver and preconditioner from setup and output costs.

still effectively used. As the number of processes grows, the efficiency decreases. This behaviour is typical of strong scaling: eventually, the local amount of work per process becomes too small to amortize the fixed overheads of communication, synchronization, and PETSc object management.

The efficiency degradation is more pronounced for the smaller grid $n = 512$. This is because, for a fixed number of processes, each MPI rank owns fewer grid points than in the $n = 1024$ case. Since the finite-difference operator uses only nearest-neighbour couplings, the computational work scales roughly with the number of local grid points, while communication is associated with the interfaces between neighbouring subdomains. When the local subdomains become small, the ratio between interface size and subdomain area increases. Consequently, communication and synchronization represent a larger fraction of the total runtime.

For the larger grid $n = 1024$, the scaling is better because each process has more local work to perform. This delays the onset of the overhead-dominated regime. In other words, the larger problem size provides enough computation to make the use of additional nodes more effective. This is the standard behaviour expected from a strong scaling study of a domain-decomposed PDE solver.

Overall, the results show that the PETSc implementation successfully solves the Poisson problem in parallel and benefits from increasing the number of MPI processes. The best scaling is observed for the larger grid, while the smaller grid reaches the strong-scaling limit earlier. This is consistent with the structure of the finite-difference discretization and with the expected communication-to-computation trade-off of distributed-memory elliptic PDE solvers.

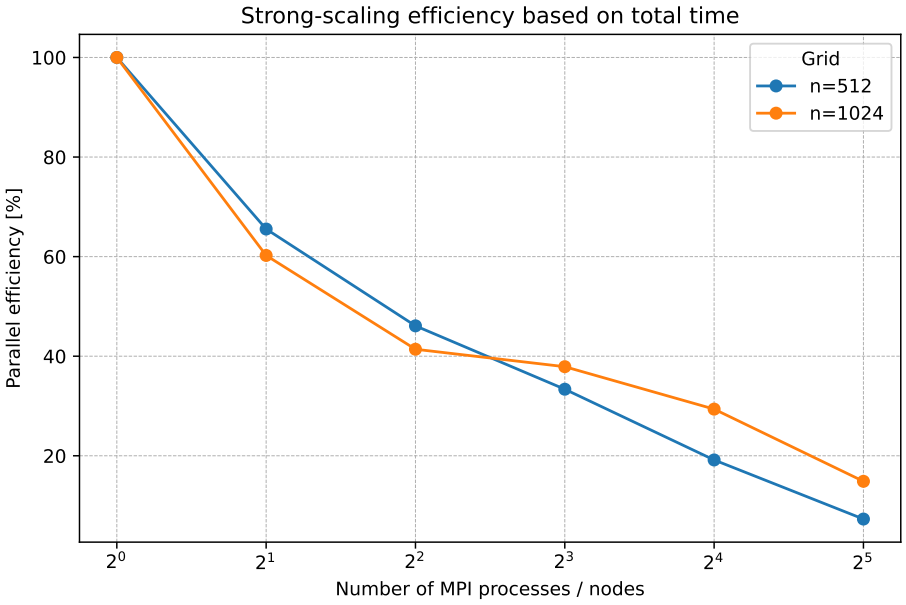


Figure 22: Parallel efficiency based on the median total time to solution. The efficiency is computed as $E_p = T_1/(pT_p)$. The decrease in efficiency at larger process counts reflects the increasing relative importance of communication and parallel overheads in the strong scaling regime.